

Data Refresh Runbook

How to keep the data behind *Where to Expand: My Project* current — what to run, how to tell it worked, and what to do when it did not.

37

DATASETS TRACKED

22

REFRESH UNATTENDED

5

NEED A MANUAL FILE

5003

LOCAL PORT

Railway

WHERE IT DEPLOYS

The short version

If you read nothing else.

```
cd Projects\tool
python refresh_web.py          → http://localhost:5003
```

Click **Refresh everything automatable** and leave it. It takes a few minutes, downloads only what has actually changed, and cannot damage anything: every file is backed up before it is rebuilt and restored automatically if the rebuild looks wrong.

When it finishes, the status column should read **ok** or **newest** for everything. Those two both mean “nothing to do”. Anything **stale** is genuinely behind and worth a look.

NOTHING IS AUTOMATIC

The tool never runs on a schedule and never refreshes anything you did not ask it to. It is a button, not a cron job.

When to run it

Most of this data changes annually. Running it more often than the sources publish achieves nothing.

WHEN	DO THIS	WHY
Monthly	Refresh everything automatable	Cheap. Skips anything unchanged, so most months it downloads nothing.
After an incentives run	Refresh <code>incentives_index</code>	The scorer reads its own summary of the incentives, not the master. Without this the two drift.
After editing an EDO	Refresh <code>edo_indexes</code>	Lead routing is a derived index. Change a territory without rebuilding and leads route to the wrong EDO, or to nobody.
Adding a property dataset	Refresh <code>orgs_with_properties</code>	Parses the dashboard's <code>properties.js</code> for the property-access bonus.
Once a year	Check the five manual sources	See section 5 . Nothing prompts you.

LEAVE “ONLY UPDATE NEWER FILES” TICKED

With it on, a refresh asks each source whether anything has actually been published since the file was last built, and skips it if not. A full sweep with nothing new downloads zero bytes and takes seconds. Untick it only when you want to force a rebuild.

Running a refresh

What each button does.

1

Start the tool

`python refresh_web.py` in `Projects\tool`, then open `localhost:5003`. It runs on your machine only. Port 5003 sits alongside the Incentives tool on 5001 and the Organizations editor on 5002, so all three can run at once.

2

Check the sources first, if you like

`Check sources only` touches every source and reports whether it answers, without downloading or rebuilding anything. Useful after a long gap, and the fastest way to find a source that has moved.

3

Refresh

`Refresh everything automatable` is the normal action. `Refresh what is stale` narrows it to datasets actually behind. Ticking rows and using `Refresh selected` does one or a few.

4

Watch the log

Each dataset prints what it decided and why – `[skip] up to date`, `[newer] source published...`, `[ok]` with record counts, or `[!] REJECTED` followed by a rollback. Cancel stops cleanly after the current step.

Everything also works from the command line, which is what to use if you want it in a script:

```
python refresh_engine.py list
python refresh_engine.py check all
python refresh_engine.py refresh auto
python refresh_engine.py refresh ca_labour county_bea
python refresh_engine.py refresh county_bea --force
```

Reading the table

Two columns are easy to misread, and both matter.

Data vintage is not the file date

The table shows both. **Data vintage** is how old the data actually is, read from the file's own provenance — hover it to see where that came from. **File written** is only when the file was last touched.

They come apart constantly. Rewriting a file does not make its contents newer, and a file rewritten this morning can hold data from 2017. Judge staleness on the vintage; ignore the file date unless you are debugging.

“stale” and “newest” are different claims

STATUS	MEANS	ACTION
ok	Current	None
newest	Past its publication cycle, but this is the newest vintage the source has	None. Refreshing cannot improve it.
stale	Genuinely behind something newer that exists	Refresh it
thin / broken	Below its record floor, or unreadable	Refresh; if it persists, see section 7

A dataset showing **newest** for a year is normal. ACS 5-year data describes a period ending well over a year ago and is still the latest anyone publishes.

Getting it live

Refreshing the data locally does not change the website. This is the part that does.

Nothing goes to GoDaddy

Worth being clear about, because it is the natural assumption. The site is on GoDaddy, but the *scoring* is not:

PIECE	LIVES ON	TOUCHED BY A REFRESH?
The intake form — <code>site_selection_intake.html</code> , <code>intake.js</code>	GoDaddy, fastlocations.ai	No
The scoring API and every data file it reads	Railway	Yes — all of it

The form on GoDaddy is a form. It posts to the API at `fastlocations-scoring-api-production.up.railway.app` (set in `intake.js`) and renders what comes back. The data lives inside the API deployment, so it ships by deploying the repository — there is no file to upload to the host.

Shipping a refresh

1 Commit and push the refreshed files

The runtime JSON is version-controlled, so a refresh shows up as a normal diff. Review it — a refresh that changes a screen input deserves a look before it ships.

2 Merge to `main`

Railway redeploys from `main`. The build installs `requirements.txt` and starts `app:app` under gunicorn.

3 Confirm it took

Open the API's health endpoint. It reports what the deployed scorer actually loaded, which is the only proof that matters:

```
https://fastlocations-scoring-api-production.up.railway.app/health

{"counties":3144,"edos":390,"states_with_incentives":65,"status":"ok"}
```

If those numbers match what `refresh_engine.py list` shows locally, the deploy carried the data. If they match the *old* values, it did not — the deploy has not run, or it ran from the wrong branch.

CHECK THE RAILWAY SETTINGS ONCE

There is no `railway.json` in the repository, so the deploy configuration lives only in the Railway dashboard. Confirm which branch it watches and whether auto-deploy is on — otherwise a merge changes nothing and the health check will tell you so.

TWO STALE FILES

`render.yaml` and `Procfile` in `Projects/` describe a Render deployment, while `intake.js` points at Railway. If Render is retired, those files are misleading rather than harmful — but do not follow them.

The five manual ones

These sources sit behind bot protection or a download form, so the tool cannot fetch them. Everything after the download is automatic.

Download the file, drop it in `tool\staging\<dataset>\`, then refresh that dataset. The tool finds it, unzips it if needed, and builds. It never deletes anything you put there.

DATASET	GET IT FROM	CYCLE
county_nri	hazards.fema.gov/nri → Data Resources → county table	Annual
county_innovation_index	statsamerica.org/innovation — the Measures export	Annual
county_land_cost	USDA NASS Census of Agriculture	5-yearly
ca_livability	StatCan CIMD catalogue page	5-yearly
ca_infrastructure	StatCan ODI — all ten GeoPackages into one folder	~2-yearly

STATSAMERICA: TAKE THE MEASURES FILE

Not the Index Values export. That one silently omits 51 Virginia county-equivalents, Fairfax among them.

When a source moves

Government download URLs move. The tool helps you find the new one; it never changes anything by itself.

1 Confirm it is actually broken

Check sources only. A source that has moved reports an HTTP error or a page that is not the file.

2 Ask it to look

Research sources searches with Brave, hands the results to Claude, and asks which is the real download. It answers with a URL, a confidence and one sentence of reasoning, then fetches the first kilobyte to check the URL actually serves bytes.

3 Approve, then verify

Nothing is saved until you click **Use this URL**. Then force a refresh of that one dataset and read the **keys usable** figure in the log.

AN APPROVED URL IS A CLAIM, NOT A FACT

Claude picks the most plausible link in the search results. It cannot tell a 2021 dataset from the 2016 edition of the same series sitting at a near-identical URL — which has happened, and produced a file with *more* records and 87% less usable coverage. The safety checks caught it. Check the numbers anyway.

When it goes wrong

Most failures are one of these five.

LOG SAYS	MEANING	FIX
REJECTED ... below the floor	The rebuild produced too few records. The source probably changed shape.	Already rolled back. Open the source and see what changed.
REJECTED ... matches only N keys	The data is real but on the wrong geography vintage.	Already rolled back. Wrong file — check the URL.
Census rejected the key	Missing, or signed up but never activated.	Click the activation link in the confirmation email.
nothing matching ... in staging	A manual dataset with no file staged.	Section 5.
could not read source headers	Source unreachable.	Retry; if it persists, section 6.

Every refresh copies the current file into `Projects\_data_backups\<dataset>\` before touching it, five deep. A rejected rebuild restores automatically, so you rarely need these — but if you want an older version back, copy it into `Projects\`.

A REJECTION IS THE TOOL WORKING

It means a bad rebuild was caught before it reached the scorer. The alternative is a model that quietly stops screening on something.

Adding a dataset

If the scorer starts reading a new file, it must be registered or the tool will not know it exists.

Add an entry to `refresh_registry.py`. The fields are documented at the top of that file; the ones that matter most are `min_records` (the floor a rebuild must clear), `key_space` (the file whose keys the scorer looks this data up by — this is what catches a wrong-geography rebuild), `cadence_days`, and `then` for anything that must be rebuilt afterwards.

Two things a new builder should do:

- Write a vintage marker — a `_meta` block with the source and its reference year — so staleness is judged on the data rather than the file date.
- Probe for the newest available vintage rather than pinning a year. A pinned year means a staleness flag can never be resolved: the builder re-fetches the same old data forever.

TWO FILES CANNOT HOLD A `_META` KEY

`county_features.json` and `ca_features.json` are the scorer's feature tables and are iterated directly, so a `_meta` key inside becomes a phantom county. Those two write provenance to a sidecar instead.

What it will not do

Worth knowing so you do not wait for it.

- **Nine files have no builder**, and that is deliberate rather than unfinished. Three are hand-curated, one is fully superseded, one is not read by the scorer at all, one belongs to the Organizations editor, one is static geometry, and one was investigated and judged not worth building. `tool/README.md` says which and why for each.
- **Some fields cannot move until 2027–28**. Canadian income, dwelling value and educational attainment are Census products with no annual series below the province — the 2026 Census releases them.
- **It does not deploy**. It refreshes the files locally; getting them live is [section 5](#).
- **It does not run on a schedule** and holds no credentials beyond the local config, which is git-ignored.